Institute of Technology of Cambodia
Department of Applied Mathematics and Statistics

TP2:
Building a Deep Neural Network, Step by Step

PRESENTATION BY GROUP 2

INSTRUCTED BY MR. VANN VISAL

Our team



SOM DARA



SOKHAL SOZANA



SONG SOLINE



SUM SAKPHEAPANCHARITH

Learning Objectives

1. Understand Forward and Backward Propagation
2. Implement Gradient-Based Learning
3. Build and Validate a Training Process

The Cost Function



Binary Cross-Entropy • Measures how well predictions match the true labels

BINARY CROSS-ENTROPY

$$J = -(1/m) \Sigma [y_i \log(a_i) + (1-y_i) \log(1-a_i)]$$

Average the loss across all m examples.

KEY SYMBOLS

- J average cost
- m number of examples
- y_i actual label: 0 or 1
- a_i predicted probability: 0–1

LOW COST

Predictions are close to the correct labels.

HIGH COST

Predictions are far from the correct labels.

TRAINING GOAL

Adjust the network to minimize the cost.

How the PyTorch Code Works

```python
def compute_cost(AL, Y, eps=1e-7):
    # Binary cross-entropy, averaged over the m examples. AL and Y have shape (m, 1).
    AL = AL.clamp(eps, 1 - eps)          # avoids log(0)
    return -(Y * torch.log(AL) + (1 - Y) * torch.log(1 - AL)).mean()
```

```python
AL_ex = torch.tensor([[0.8], [0.9], [0.4]])
Y_ex = torch.tensor([[1.0], [1.0], [0.0]])
print(f"cost = {compute_cost(AL_ex, Y_ex).item():.4f}")
```

1. **Define**

AL = predictions; Y = true labels; eps = tiny safety value.

2. **Clamp**

Keep AL between 0.0000001 and 0.9999999 to avoid log(0).

3. **Calculate**

Select the correct class probability, take its log, then average the loss.

4. **Test**

Three predictions: 0.8, 0.9, 0.4; labels: 1, 1, 0.

5. **Output(Average cost)** = 0.2798

Backward Propagation

The four equations for layer l (lecture slide "The equations for layer l"):

		shape
$d\mathbf{Z}^{[l]} = d\mathbf{A}^{[l]} \odot g^{[l]\prime}(\mathbf{Z}^{[l]})$	elementwise product	$(m, n^{[l]})$
$d\mathbf{W}^{[l]} = \frac{1}{m}\, \mathbf{A}^{[l-1]\top} d\mathbf{Z}^{[l]}$		$(n^{[l-1]}, n^{[l]})$
$d\mathbf{b}^{[l]} = \frac{1}{m} \sum_i d\mathbf{Z}^{[l]}_{i,:}$	sum over the examples (rows)	$(1, n^{[l]})$
$d\mathbf{A}^{[l-1]} = d\mathbf{Z}^{[l]}\, \mathbf{W}^{[l]\top}$	passed to layer $l-1$	$(m, n^{[l-1]})$

A useful habit: **every gradient has the shape of the thing it is the gradient of.** $d\mathbf{W}$ has the shape of $\mathbf{W}$, $d\mathbf{A}^{[l-1]}$ has the shape of $\mathbf{A}^{[l-1]}$.

The Linear Backward Function

```python
def linear_backward(dZ, cache):
    A_prev, W, b = cache
    m = A_prev.shape[0]                          # examples are rows
    dW = A_prev.T @ dZ / m                       # (n_prev, n_l)
    db = dZ.sum(dim=0, keepdim=True) / m         # (1, n_l)
    dA_prev = dZ @ W.T                           # (m, n_prev)
    return dA_prev, dW, db
```

```python
torch.manual_seed(2)
dZ = torch.randn(3, 2)                           # same shape as Z in Part 3.1
dA_prev, dW, db = linear_backward(dZ, (A_prev, W, b))
print("dA_prev", tuple(dA_prev.shape), "same as A_prev", tuple(A_prev.shape))
print("dW     ", tuple(dW.shape), "same as W     ", tuple(W.shape))
print("db     ", tuple(db.shape), "same as b     ", tuple(b.shape))
```

1. **Unpack the cache**

Retrieve A_prev (previous activations), W (weights) and b (bias).

2. **Count examples**

m = A_prev.shape[0] gives the number of training examples (rows).

3. **Weight gradient**

$dW = A_prev^T \times dZ / m$ → how the weights contributed to the cost.

4. **Bias gradient**

db = sum dZ across rows / m → average gradient for each bias.

5. **Pass gradient backward**

$dA_prev = dZ \times W^T$ → sends the gradient to the previous layer.

Activation

main formula: $dZ^{[l]} = dA^{[l]} \odot g'^{[l]}(Z^{[l]})$

activation derivatives:

$$\text{ReLU}: \quad g'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

$$\text{Sigmoid}: \quad g'(z) = \sigma(z)(1 - \sigma(z))$$

During forward propagation, activation functions transform Z into A.

During backward propagation, we calculate how the gradient passes through the activation function.

```python
def relu_backward(dA, cache):
    Z = cache
    # g'(Z) = 1 if Z > 0 else 0
    return dA * (Z > 0).float()

def linear_activation_backward(
    dA, cache, activation
):
    lin_cache, act_cache = cache
    if activation == "relu":
        dZ = relu_backward(dA, act_cache)
    return linear_backward(dZ, lin_cache)
```

$$dA \longrightarrow \text{Activation Derivative} \longrightarrow dZ$$

Starting the Backward Pass

The Initial Problem

Backpropagation needs an initial gradient.

$$dA^{[L]} = -\left(\frac{Y}{A^{[L]}} - \frac{1-Y}{1-A^{[L]}}\right)$$

Algebraic Shortcut

$$dZ^{[L]} = A^{[L]} - Y$$

$$\text{Output} \rightarrow \text{Hidden Layers} \rightarrow \text{First Layer}$$

$dW^{[l]}$ How much the weights contributed to the error

$db^{[l]}$ How much the biases contributed to the error

$dA^{[l-1]}$ Gradient passed to the previous layer

```python
def L_model_backward(AL, Y, caches):
    grads = {}
    L = len(caches)

    # Output layer: dZ[L] = AL - Y
    lin_cache, _ = caches[L - 1]
    dA, grads[f"dW{L}"], grads[f"db{L}"] = \
        linear_backward(AL - Y, lin_cache)

    # Hidden layers (from L-1 down to 1)
    for l in range(L - 1, 0, -1):
        dA, grads[f"dW{l}"], grads[f"db{l}"] = \
            linear_activation_backward(
                dA, caches[l - 1], "relu"
            )
    return grads
```

Verification via PyTorch Autograd

```python
def compare_with_autograd(fwd, bwd, params, X, Y):
    # 1. Autograd reference setup
    p = {k: v.clone().requires_grad_()
         for k, v in params.items()}
    AL, _ = fwd(X, p)
    compute_cost(AL, Y).backward()

    # 2. Custom manual backprop
    AL, caches = fwd(X, params)
    grads = bwd(AL, Y, caches)

    # 3. Measure differences
    for k in params:
        diff = (grads["d" + k] - p[k].grad).abs().max
    ().item()
        print(f"d{k:3s} max diff: {diff:.1e}")
```

Expected Output

```
dW1 max diff:  1.5e-08
db1 max diff:  7.5e-09
dW2 max diff:  1.5e-08
db2 max diff:  0.0e+00
```

Validation

Main formula: $dZ^{[l]} = dA^{[l]} \odot g'^{[l]}(Z^{[l]})$

Activation derivatives:

- ReLU : $g'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$
- Sigmoid : $g'(z) = \sigma(z)(1 - \sigma(z))$

Parameter Updates & Optimization Step

Gradient Descent Update

$$\mathbf{W}^{[l]} \leftarrow \mathbf{W}^{[l]} - \alpha \, d\mathbf{W}^{[l]}$$

$$\mathbf{b}^{[l]} \leftarrow \mathbf{b}^{[l]} - \alpha \, d\mathbf{b}^{[l]}$$

```python
def update_parameters(params, grads, lr):
    return {
        k: params[k] - lr * grads["d" + k]
        for k in params
    }
```

```python
# --- ONE COMPLETE TRAINING STEP ---
# 1. Forward Pass
AL, caches = L_model_forward(X_small, params_small
    )
cost_before = compute_cost(AL, Y_small).item()

# 2. Backward Pass
grads = L_model_backward(AL, Y_small, caches)

# 3. Update Parameters (alpha = 0.5)
params_after = update_parameters(
    params_small, grads, lr=0.5
)

# 4. Evaluate Cost Reduction
AL_after, _ = L_model_forward(X_small,
    params_after)
cost_after = compute_cost(AL_after, Y_small).item
    ()

# Result: cost_before = 0.6001 | cost_after =
    0.5828
```

Key Takeaways

Cache Management

Store input activations and linear outputs during the forward pass to compute gradients efficiently.

Numerical Stability

Use $dZ^{[L]} = A^{[L]} - Y$ (sigmoid + BCE) to avoid division and prevent `NaN` errors.

Verification with Autograd

Compare manual gradients with PyTorch autograd to verify numerical correctness.

Thank you!

Any questions?